

代数函数与图像语言

第007单元 把一个公式说成可以检查的过程

看到公式 $y = 2x + 1$ ，你能把某个输入代进去计算；但如果把x换成别的单位，或给输出取对数，再把多条结果平均，原来的解释还成立吗？本讲把公式当成输入、计算和输出之间的一份明确约定。我们从一元方程、函数和坐标图开始，再建立指数、对数、求和与复合的语言。

先修001与003。只要求理解四则运算、预测误差和基本Python函数，不借用后面的向量、矩阵或微积分。所有图都是标量函数的示意；学完应能写出输入允许的范围、单位和逐步计算，并说明图形支持哪些判断、不能证明哪些判断。

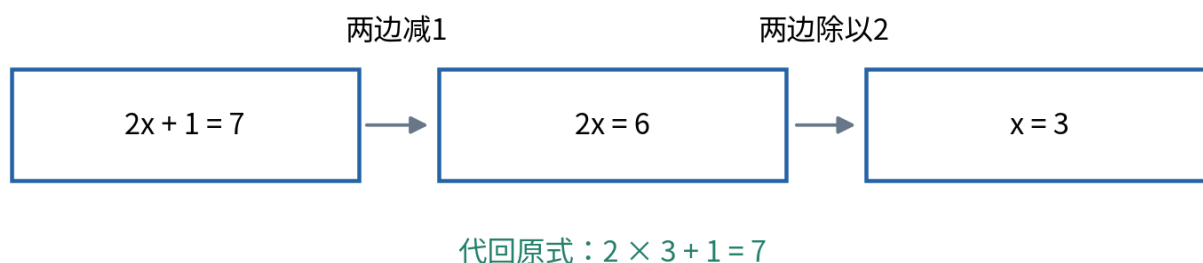
核心例子仍然是一个合成校准关系。简单直线帮助我们认识参数与输入；重复乘2帮助认识指数；反过来问“乘了几次2”得到对数；最后用两个损失列表说明：对每个数取对数再平均，可能改变模型排名。

一 字母代表哪个数

一个表达式由数、字母和运算组成，例如 $2x+1$ 表示把x乘2再加1。数学里紧挨的 $2x$ 表示乘法；Python必须写成 $2 * x$ 。x是本次可以变化的输入，2与1在这条规则里固定。把规则一般化为 $ax+b$ 时，a和b是我们选择或估计的参数。

本讲的x、a、b、y都是单个实数。实数包括整数、分数、小数和其他可以落在连续数轴上的数；暂不讨论复数。字母不是新的物理量，也不自动规定单位。若x是旋钮格数，y是读数单位，a的单位应为“读数单位/格”，b的单位应与y相同。

等式表达两边相等的关系。写 $2x+1=7$ ，就是要求找到让两边相等的x。两边同时减1，得到 $2x=6$ ；再同时除以2，得到 $x=3$ 。每一步都对两边做同样且合法的操作，所以保持了原来允许的解。



一般 $ax+b=c$ ： $a \neq 0$ 才能除以a； $a=0$ 必须另行判断

图1 方程求解与代入核对。减去同一个数、除以同一个非零数保留相等关系；最终还要把解放回原式。

把 $x=3$ 代回原式，左边 $2 \times 3 + 1 = 7$ ，与右边一致，这叫验算。若某一步除以可能为0的量，就必须先处理零的情况。一般方程 $ax+b=c$ 在 a 不为0时有解 $x=(c-b)/a$ ； $a=0$ 且 $b=c$ 时，每个允许的 x 都满足； $a=0$ 且 b 不等于 c 时没有解。

这三个情况说明，公式不能脱离条件使用。程序也应区分唯一解、无解和多解，不应该遇到 $a=0$ 就随意输出0。我们将在实验中给这三种情况分别写测试。

Python中的 $x = x + 1$ 是赋值：先用旧 x 计算右侧，再把结果命名为新的 x 。它不是声称某个实数等于自己加1。数学等式与程序状态更新相似的外观，不能代替各自语义。

二 函数给每个允许输入一个确定输出

函数由允许的输入集合和一条明确对应规则组成。在定义域内，每个输入应对应一个确定输出。把校准规则命名为 f ，写成 $f(x) = 2x + 1$ ，意思是 f 接收 x ，输出 $2x+1$ 。括号不是乘法， $f(3)$ 表示“把3交给函数 f ”，计算结果是7。

定义域是允许交给函数的输入范围。数学表达式 $2x+1$ 对任意实数都能计算，但合成装置也许只允许旋钮在0到5格。后者是任务额外限制。能算出 $f(100)=201$ ，不证明设备真的支持100格或这条模型在那里准确。

输出实际可能取到的值称为值域；我们先不把它与另行指定的输出集合混为一谈。若输入限定为0到5， $f(x)=2x+1$ 的输出从1到11。输入不能超出范围时，输出也不应被解释为任意实数。

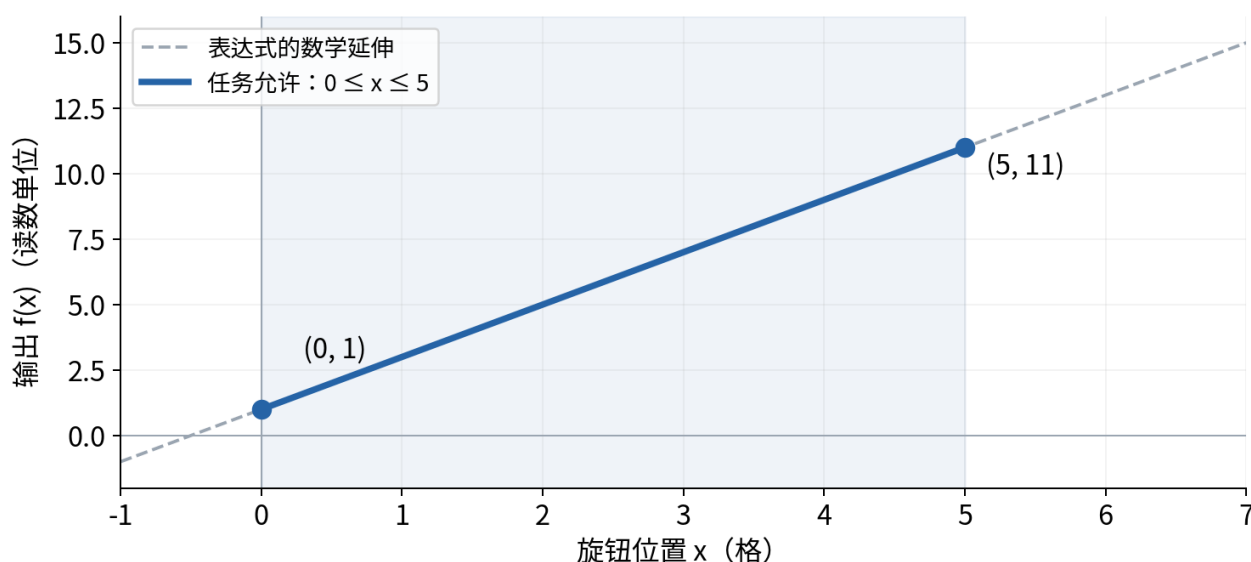


图2 相同表达式可以配不同定义域。模型可计算的范围与真实任务允许或有证据支持的范围，需要分别写明。

Python函数的参数和返回值可以实现数学函数的一部分，但任意程序函数未必是这里的数学函数。例如程序读取当前时间或修改全局状态，同一显式输入可能得到不同结果。若希望它表达一个确定的数学关系，应把有关状态也作为约定的一部分。

三 坐标图把对应关系放到位置上

横轴表示输入，纵轴表示输出。一个点(2,5)表示 $x=2$ 时 $y=5$ ，先读横向位置，再读纵向高度。作图从表格开始： x 为0、1、2、3时， $f(x)$ 分别为1、3、5、7。标出这四个点，再结合明确的函数形式画出直线。

这里把图像为直线的关系系统称为直线关系。严格地说， $ax+b$ 在 b 不为0时叫仿射函数；保持原点的 ax 才是后续线性代数语境下的线性函数。本讲的 $2x+1$ 是仿射关系，不需要先懂线性代数。

本例输入每增加1，输出都增加2。这个固定变化率使点落在一条直线上。 a 决定输入增加一个单位时输出改变多少， b 是输入0时的输出。 a 为正时向右看上升， a 为负时下降， $a=0$ 时是水平线。此处用有限差值解释，不预先要求导数。

实际数据点不一定都落在模型曲线上。把数据点画成实心圆、模型画成线，有助于区分观测和拟合。仅靠少量点连出的折线也不能证明点与点之间遵循同样规律；图中的连续线来自我们采用的函数假设。

坐标轴比例会改变视觉倾斜程度。把横轴拉宽，同一关系看起来更平；把纵轴缩短，也会改变视觉印象。因此比较变化率应读数值和单位，不凭线在页面上“陡不陡”下结论。

四 换单位时参数也要换

假设输入是以厘米记录的长度，规则为 $y=2x+1$ 。把同一长度改成毫米，数值变为原来的10倍。若原来 $x=3$ 厘米，对应30毫米；直接把30放进旧式会得到61，而原预测是7。这不是设备突然变了，而是输入表示与系数不匹配。

令新的毫米数值为 u ，则原厘米数值为 $u/10$ 。代回原式得到 $y=2(u/10)+1=0.2u+1$ 。用 $u=30$ 计算，结果仍是7。单位转换既改变输入数值，也改变与它相乘的系数。

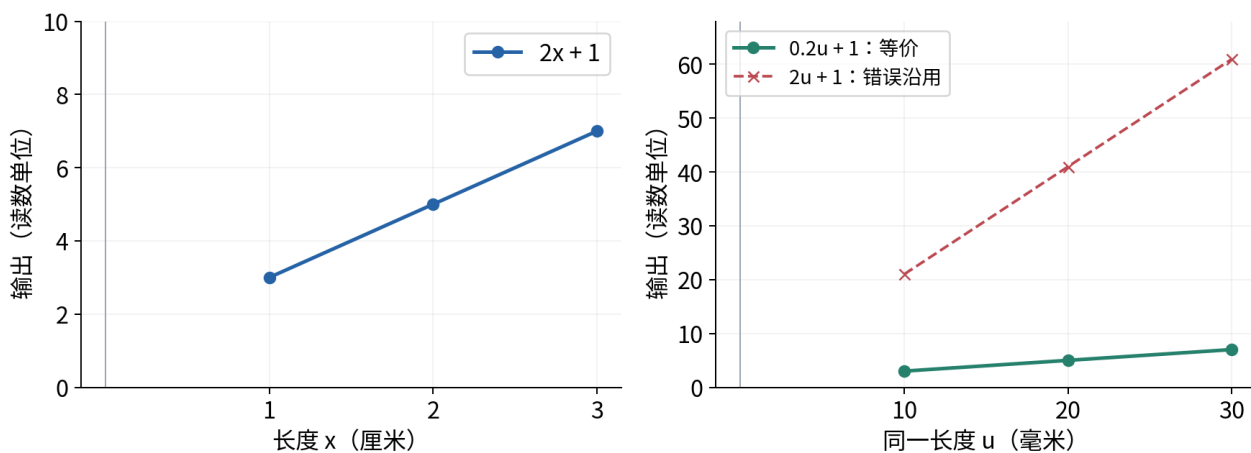


图3 同一对象换单位后，输入数字放大10倍，系数应缩小10倍，预测保持一致。直接沿用旧系数会改变模型含义。

加法两项需要可比较的单位，例如长度加长度。指数与对数的输入通常使用无量纲数或相对于明确参考值的比值。对“3厘米”直接取对数却不说明参考单位，会使结果随着你写厘米还是毫米改变；改写为长度除以1厘米的比值，才明确了所用基准。

五 从重复乘法认识指数

平方 x^2 表示 x 乘自己一次，例如 $3^2=9$ 。一般正整数 n 次幂 a^n 是 n 个 a 相乘。这里幂的底数是 a ，指数是 n 。Python 用 `a ** n`，符号 $^$ 在 Python 不是幂运算。

以底数 2 为例， $2^1=2$ 、 $2^2=4$ 、 $2^3=8$ 。为了保持 $2^{m+n} = 2^m 2^n$ 的关系，零次幂定义为 1，负一次幂为 $1/2$ ，负二次幂为 $1/4$ 。这个解释采用非零底数；不能把关于负指数的规则直接套到 0 上。

底数 2 的指数函数写作 $g(x) = 2^x$ ，变量出现在指数位置，而不是 $2x$ 的乘法位置。整数输入每增加 1，输出乘 2。 x 从 0 到 4 时输出为 1、2、4、8、16，和直线 1、3、5、7、9 的变化方式明显不同。

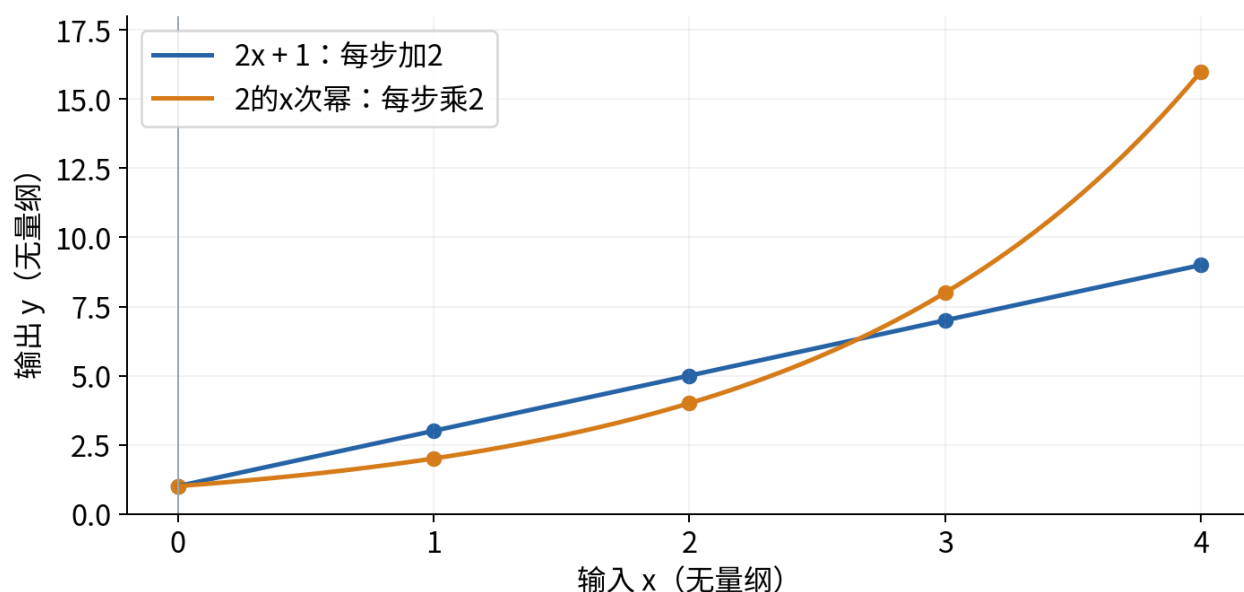


图4 固定增加量与固定倍数是不同关系。直线每步加2，指数每步乘2；有限区间的图不能替代对全部输入的论证。

在正底数下可以把指数扩展到分数与实数。例如 $2^{1/2}$ 是平方后等于 2 的正数，约 1.414；这让两个半步的乘法因子组合成完整一步的 2。实数指数的严格构造需要更深入的数学，本讲使用它的标准定义与运算规律，并用有限例子检查，不声称画图完成了证明。

当底数 a 大于 1 时， a 的 x 次幂随 x 增大而增大； $0 < a < 1$ 时则减小。我们把一般实指数函数的底数限定为正数，避免负底数与非整数指数的实数定义问题。自然指数使用特殊底数 e ，约 2.718281828；程序用 `math.exp(x)` 计算 e 的 x 次幂。为什么这个底数在微积分中特别方便，会在 010 与后续单元解释。

还要注意负号与括号。数学里的 $(-2)^2 = 4$ ，而 $-2^2 = -4$ 通常表示先算 2^2 再取负。Python 也将 `-2 ** 2` 解释为 $-(2**2)$ ，所以希望负数作为底数时应写括号。[2]

六 对数是在问指数是多少

已知 $2^3=8$ ，反过来问“2 的几次幂等于 8”，答案是 3。写作 $\log_2 8 = 3$ 。一般地， $\log_a x = u$ 与 $a^u = x$ 表达同一个关系，条件是 $a > 0$ 、 a 不等于 1、 $x > 0$ 。 a 称为对数底数， x 是被取对数的正数。

底数不能为1，因为1的不同幂都等于1，不能唯一反推出指数。 x 不能是0或负数，因为正底数的实指数结果始终为正。在实数范围内 $\log(0)$ 没有有限值， $\log(-1)$ 也不是一个实数；程序应明确拒绝，不能顺手填成0。

底数2时， x 为1/4、1/2、1、2、4、8，对数分别是-2、-1、0、1、2、3。对数为负不表示输入为负：1/2是正数，只因它小于1，所以需要负指数才能由2得到它。

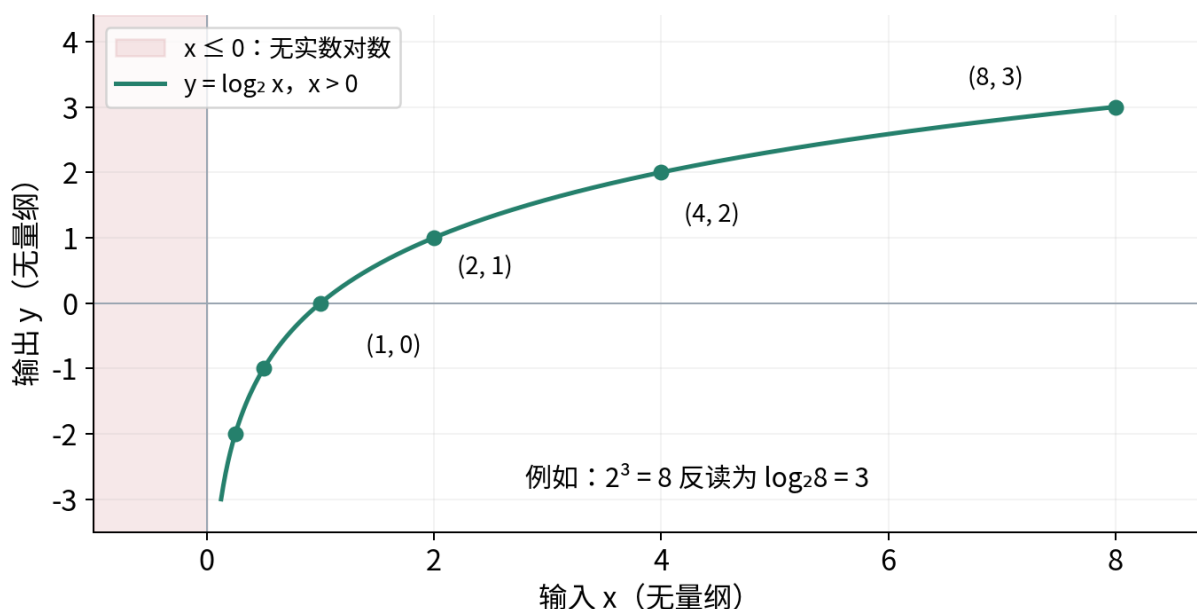


图5 对数函数只接收正输入。指数与对数把同一关系反向读取；数轴上的0不是一个可以补入的普通对数点。

底数大于1时，对数随输入增大而增大；底数在0与1之间时方向相反。常用的以e为底的对数写作 $\ln$ ，Python的 $\text{math.log}(x)$ 默认计算这一种； $\text{math.log2}(x)$ 以2为底， $\text{math.log10}(x)$ 以10为底。只写 $\log$ 却不约定底数，可能导致数值和单位解释不一致。[1][3]

换底公式为 $\log_a x = \ln x / \ln a$ 。它使我们可以从自然对数得到别的底数。下一节先解释对数的幂律，再推导此式，避免把它当成另一个没有来由的计算按钮。

七 为什么乘积变成对数的和

令 $u = \log_a x$ 、 $v = \log_a y$ ，那么 $x = a$ 的 u 次幂、 $y = a$ 的 v 次幂。两者相乘得到 a 的 $u+v$ 次幂，因此 $\log_a(xy) = u+v$ ，也就是 $\log_a x + \log_a y$ 。这里 x 和 y 都必须为正，底数满足上一节条件。

用具体数字验证： $\log_2(4 \times 8) = \log_2 32 = 5$ ，而 $\log_2 4 + \log_2 8 = 2 + 3 = 5$ 。除法同理给出 $\log_a(x/y) = \log_a x - \log_a y$ 。对数把乘法结构转换成加法结构，有时方便整理多个正因子的乘积。

它不把加法直接变成对数相加。 $\log_2(4+4) = \log_2 8 = 3$ ，而 $\log_2 4 + \log_2 4 = 4$ 。把正确的乘积公式误用到和上，是一种看起来整齐却立即可被小例子推翻的符号错误。

再看幂律。令 $u = \log_a x$ ，所以 $x = a$ 的 u 次幂。对任意实数 r ，正底数指数的运算规律给出 $(a^u)^r = a^{ur}$ 。于是 x 的 r 次幂所对应的指数就是 ur ，得到：

$$\log_a(x^r) = r \log_a x, \quad x > 0, a > 0, a \neq 1.$$

例如 $\log_2(4^3)=\log_2 64=6$ ，右侧 $3\log_2 4=3\times 2=6$ 。限定 $x>0$ 很重要：虽然 $(-2)^2=4$ 可取对数， $\log_2(-2)$ 却没有实数值，不能用幂律把它变成 $2\log_2(-2)$ 。

现在推导换底。令 $u=\log_a x$ ，依定义有 $x=a^u$ 。两边取自然对数，利用刚才的幂律得到 $\ln x=u \ln a$ 。由于 a 不为1， $\ln a$ 不为0，可以除以它，得到：

$$u = \frac{\ln x}{\ln a}, \quad \text{即} \quad \log_a x = \frac{\ln x}{\ln a}.$$

这段推导中的每一步都要求 x 和 a 为正，且 a 不为1。换成任何另一个合法对数底数，也能用同样过程推导。[4] 这些是实数数学的恒等式；在程序中先算 x 的 r 次幂或 xy 可能溢出，对公式两边分别计算得到的浮点数也不必逐位相同。实验用小数与明确误差容限核验。

八 对整体取对数与逐项取对数

若比较两个正数 A 和 B ，底数大于1的对数保持大小顺序： $A<B$ 时 $\log A<\log B$ 。因此对同一个正的整体目标应用这种单调变换，最优选择的顺序不会因这一步改变。但先变换各项、再把它们加起来，通常已经构造了另一个目标。

设两个方案的两条正损失分别为 $A=[1,9]$ ， $B=[4,4]$ ，损失越小越好。原平均损失 A 为5， B 为4，所以原目标选 B 。若先取以2为底的对数再平均， A 为 $(0+\log_2 9)/2$ ，约1.585； B 为 $(2+2)/2=2$ ，于是新目标选 A 。

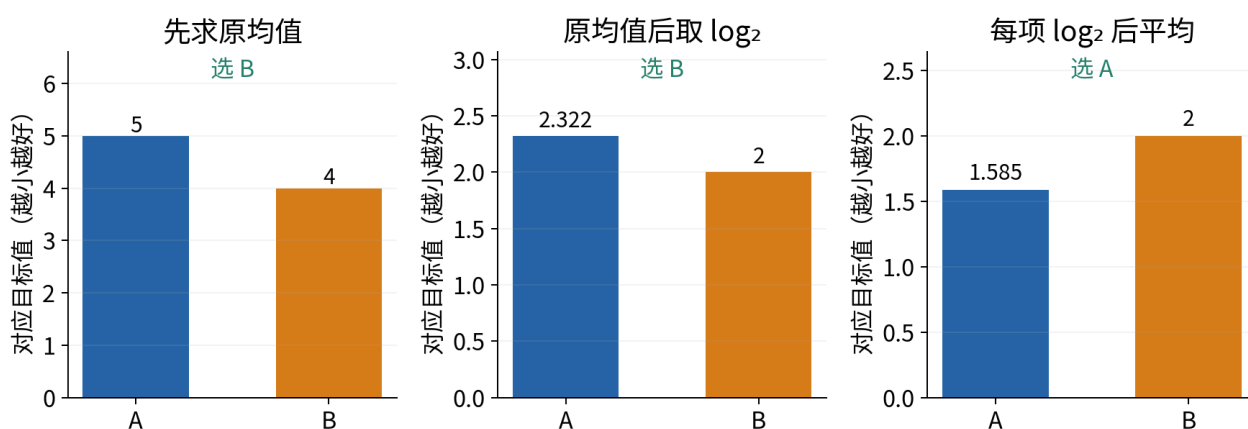


图6 两种先后顺序可能改变排名。对平均值整体取对数保留原顺序；逐项取对数再平均改变了怎样汇总各条损失。

若先算原均值再取对数， A 是 $\log_2 5$ 约2.322， B 是 $\log_2 4=2$ ，仍然选 B 。问题不在对数“有没有用”，而在你到底对哪个对象做了什么运算。改变目标有时有明确建模理由，但需要说明新目标强调什么，不能因为式子更好算就说所有评价等价。

这个反例也能直接从乘积解释。对两条正损失 p 、 q ，逐项取 $\log_2$ 再平均等于 $\frac{1}{2}\log_2(pq) = \log_2 \sqrt{pq}$ 。其中正平方根 $\sqrt{pq}$ 叫这两个数的几何平均值。 A 的几何平均为3， B 为4，所以第三种目标选 A 。它比较的是另一种汇总方式，不是原算术平均的换个写法。这里不需要先学习概率或不等式理论。

上述保序结论只适用于同一正整体以及大于1的底数。若改用底数 $1/2$ ，对数递减，大小方向就反转；若仍把分数最小当作最好，就不能声称选择不变。

后续概率模型常出现对数似然，那里会先写清概率乘积、样本假设与优化目标，再讨论对数变换。现在先掌握这个反例，避免在没有定义对象时机械套公式。

九 求和符号把循环写短

001已经用平均误差接触过求和。现在把它展开： $\sum_{i=1}^3 a_i$ 就是 $a_1+a_2+a_3$ 。 i 是逐个变化的编号，1和3指定从哪里开始、到哪里结束； a_i 是编号对应的数，不是 a 乘 i 。换一个不冲突的编号字母，不会改变总和。

若 $a_1=2$ 、 $a_2=4$ 、 $a_3=6$ ，和为12，平均为 $12/3=4$ 。Python列表从位置0开始，因此同样三个数通常用`values[0]`、`values[1]`、`values[2]`。数学编号从1开始只是本课程的记号约定，不能直接照抄成Python索引1到3。

常数也需要逐项累加。 $\sum_{i=1}^3 (a_i + 1)$ 等于 $(a_1+1)+(a_2+1)+(a_3+1)$ ，所以是在原和12上加3，得到15，而不是只加一次1得到13。将求和号外的操作移进或移出时，应先展开一个小例子。

十 复合函数的顺序

令 $f(x)=2x+1$ ， $g(x)=x^2$ 。 $f(g(x))$ 表示先把 x 交给 g ，再把 g 的输出交给 f 。 $x=2$ 时， $g(2)=4$ ， $f(4)=9$ 。反过来 $g(f(x))$ 先得到 $f(2)=5$ ，再平方得25。两种复合都定义良好，结果不同。

$f(g(2))$



$g(f(2))$



图7 复合的箭头顺序不能随意交换。括号内先算，再把中间结果交给外层；这为后续多层网络与链式法则做准备。

复合还要求内层输出落在外层的定义域。若外层是 $\log_2$ ，内层是 $x-1$ ，则 $\log_2(x-1)$ 要求 $x-1>0$ ，也就是 $x>1$ 。先前“ x 本身为正”不够，例如 $x=0.5$ 虽然正，内层结果却是-0.5，不允许取实数对数。

本讲只追踪数值与允许范围，不提前求导。以后计算一串变换怎样随输入变化时，会沿着同样的计算顺序建立链式法则。

十一 分段规则把条件也写进函数

一个函数可以在不同输入范围采用不同表达式。例如 x 小于0时输出0， x 大于或等于0时输出 x 。这给所有实数输入一个确定的非负输出，常写成 $\max(0,x)$ ，也是后续会用到的ReLU函数。

```
def rectifier(x):
    if x < 0:
        return 0
    return x
```

在 $x=-2$ 、 0 、 3 时，输出依次是 0 、 0 、 3 。边界 0 必须写清；若两个分支都不包含 0 ，函数在这个位置就缺少定义；若分支重叠且给不同答案，又会产生歧义。本例在边界上两种表达式都给 0 ，但程序仍保持明确的分支规则。

用分段记号写出来是：

$$r(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

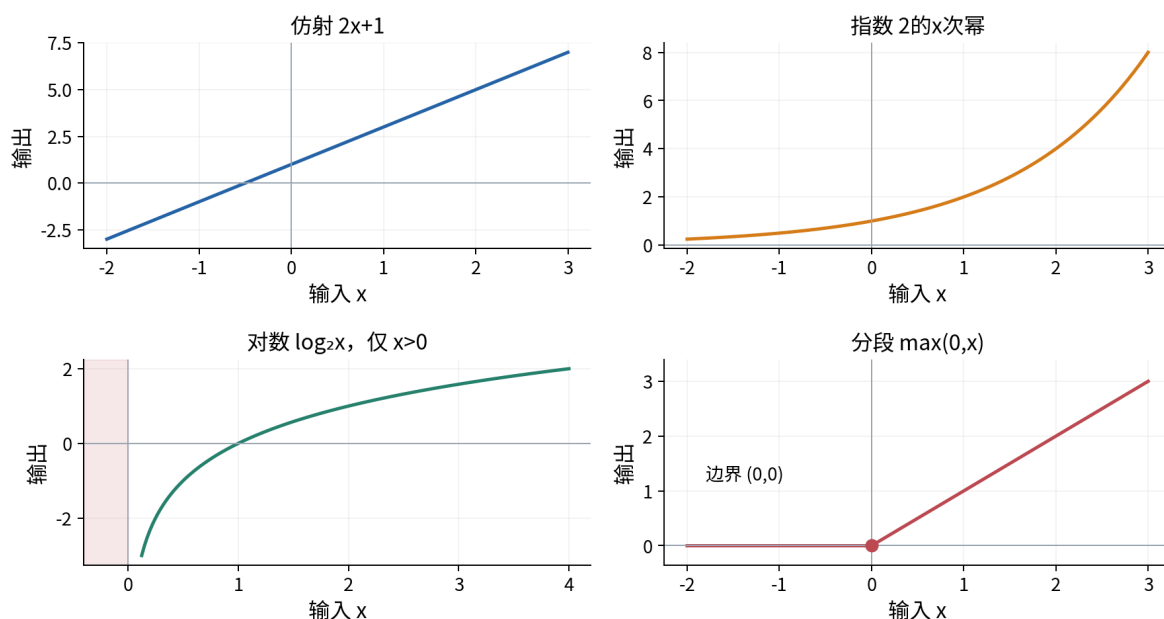


图8 四种输入输出规律。各小图均用普通坐标，横轴为无量纲输入；对数仅画正输入，分段规则在 0 处有定义。直线每步的增加量固定，指数的倍数固定，对数反读指数，分段函数必须先判断输入范围。

图形在 0 处转折，不代表函数值没有定义。函数是否可求导是另一个问题，将在010讨论。不要把“看起来有尖角”“数值不连续”“没有定义”当成同一种情况。

十二 从数学约定到程序检查

数学上的 2 的 x 次幂对每个实数 x 都有正值，但计算机的浮点表示有范围限制。非常大的正指数可能溢出，非常大的负指数可能被舍入成 0 。它们与 $\log_2(-1)$ 在实数中没有定义是两类不同问题，不能都填成“函数值为 0 ”。本实验把前者报告为`ArithmeticError`，把非法对数输入报告为`ValueError`；不把这些异常当作数学反例。[1]

配套程序只用于有限、适度大小的标量。它拒绝NaN、无穷大、字符串与布尔值，检查非有限输出以及指数下溢为0的情况；并不声称解决全部舍入、消去误差或极端规模问题。特别是比较a是否为0，是检查传入的数是否恰为0，不是在判断带测量误差的斜率“足够接近0”。用容差把小a当作0会改变问题，需要另外给出应用理由。

实验中的CSV保留对数未定义位置的空白，并额外记录outside_real_domain。这样读者能区分“没有定义”“真的等于0”和“没有运行”。图形只显示有限窗口；证明公式仍要回到对象、假设与合法操作。

十三 练习

1. 分别求解 $2x+1=7$ 、 $0x+3=3$ 、 $0x+3=4$ ，说明每步操作的条件。
2. 对输入限制在0到5的 $f(x)=2x+1$ ，写出定义域与值域。计算 $f(100)$ 是否证明装置在100格可用？
3. 把厘米输入改成毫米，推导等价系数，并对同一物理长度验算。
4. 列出2的-2、-1、0、1、2、3次幂；核对 $(-2)^2$ 与 -2^2 在Python中的写法和结果。
5. 算 $\log_2(1/4)$ 、 $\log_2 1$ 、 $\log_2 8$ ，写出实数对数对输入与底数的限制。
6. 用4和8推导并核对乘积对数公式，再给出“和的对数等于对数的和”的反例。
7. 完整比较损失列表 $A=[1,9]$ 与 $B=[4,4]$ 的三种目标：原均值、均值后取 $\log_2$ 、逐项 $\log_2$ 后平均。解释排名为何可能变化。
8. 对 $f(x)=2x+1$ 与 $g(x)=x^2$ 计算两种复合在 $x=2$ 时的值，再写 $\log_2(x-1)$ 的定义域。
9. 展开三项求和 (a_i+1) ，核对与“求和后只加1”的区别。给分段rectifier在负数、0与正数上各写一个测试。

参考资料

[1] Python3.12官方文档，[math](#)，指数、对数与定义域错误。[2] Python3.12语言参考，[Operator precedence](#)，幂与负号的解析。[3] OpenStax作者教材，[College Algebra 2e](#) 第6.3节，对数函数定义与条件。[4] OpenStax作者教材，[College Algebra 2e](#) 第6.5节，乘积、商、幂与换底公式。例子、图、程序与目标变换反例独立编写。